Fluxo de Desenvolvimento e Infraestrutura — Suprimentos Dream

Projeto: Comparador de cotações Suprimentos Dream **Produção:** https://suprimentos-dream-production.up.railway.app **Repositório:** https://github.com/TecnologiaDreamFactory/suprimentos-dream **Documento criado em:** 21/04/2026

Sumário

1. Visão geral da arquitetura
2. Stack técnica
3. Mapa de pastas — quem faz o quê
4. Fluxos de dados da aplicação
5. Integração com OpenAI (determinístico vs IA)
6. Infraestrutura de produção
7. Pipeline CI/CD (como uma alteração chega em produção)
8. Variáveis de ambiente
9. Rotas HTTP expostas
10. Segurança (camadas aplicadas)
11. Persistência e armazenamento
12. Observabilidade (logs, métricas, readiness)
13. Testes automatizados
14. Fluxo de desenvolvimento recomendado para o time
15. Riscos conhecidos e evoluções sugeridas

---

1. Visão geral da arquitetura

```
+--------------------+          +----------------------------+
|    Browser Web     |          |  Integrações HTTP externas  |
|  (public/index)    |          |  (Workers, cURL, Postman)   |
+---------+----------+          +--------------+--------------+
          |                                     |
          |  HTTPS                              |  HTTPS
          +-----------------+------------------+
                            |
                            v
        +--------------------------------------+
        |   Railway Edge (TLS, roteamento)      |
        +------------------+-------------------+
                           |
                           v
        +--------------------------------------+
        |   Container Node.js                   |
        |   server.js (Express)                 |
        |   porta = $PORT (injetada)            |
        +---+----------+-----------+------------+
            |          |           |
            v          v           v
       +--------+  +---------+  +-----------------+
       | rfq/   |  |  ai/    |  | public/ (UI)    |
       |pipeline|  | openai  |  | index.html + js |
       +---+----+  +----+----+  +-----------------+
           |            |
           v            v
    +-----------+  +--------------------+
    | Filesystem|  | OpenAI API         |
    | efêmero   |  | (Responses API)    |
    | do RW     |  | api.openai.com     |
    +-----------+  +--------------------+
```

Pontos-chave

- **Um único processo Node.js** roda toda a aplicação (API + serving do frontend estático).
- **Frontend** é servido como arquivos estáticos de `public/` pelo próprio Express — não há build step nem framework SPA.
- **Processamento determinístico** (parse de Excel, consolidação, ranking) sempre executa. A **OpenAI é opcional** — entra como enriquecimento.
- **Persistência em disco local** para histórico/snapshots de lotes. No Railway, isso é efêmero (ver seção 11).

2. Stack técnica

Camada	Tecnologia	Versão / nota
Runtime	Node.js	16+ (Railway detecta via Nixpacks; usamos a LTS corrente)
Framework HTTP	Express	4.18
Upload de arquivos	Multer	1.4.5-lts.1 (armazenamento em memória)
Parse Excel	SheetJS (`xlsx`)	0.18
Export Excel	ExcelJS	4.4
Extração PDF	pdf-parse	2.4
Extração DOCX	mammoth	1.12
Imagens em XLSX	@resvg/resvg-js	2.6 (gráficos de pizza 3D)
HTTP client	axios	1.7
CORS	cors	2.8
Dotenv	dotenv	17.3
IA	SDK `openai` (Responses API + JSON estruturado)	4.104
Frontend	HTML + JS vanilla	sem framework, sem build
Testes	`node:test` (runner nativo)	Node 18+
Versionamento	Git + GitHub	—
Hospedagem	Railway (Nixpacks)	—

Sem banco de dados relacional. Toda a persistência operacional (histórico, snapshots, exports) é em arquivos JSON/XLSX no filesystem. Estado transitório (rate-limit, tokens de download) vive em memória.

3. Mapa de pastas — quem faz o quê

```
compras-suprimentos-dream/
├── server.js                     # Ponto de entrada. Configura Express, rotas, middlewares, shutdown.
├── package.json                  # Dependências + script "start" (usado pelo Railway).
├── .env.example                  # Referência de variáveis (sem valores reais).
├── .env                          # LOCAL apenas — ignorado pelo git.
│
├── public/                       # Frontend estático
│   ├── index.html                # Interface web completa (1478 linhas)
│   └── js/
│       ├── batch-result-ui.js    # Renderização do resultado de lote
│       └── ai-thinking-gsap.js   # Animações de feedback visual durante a IA
│
├── ai/                           # Camada de integração com OpenAI
│   ├── openaiClient.js           # Cliente (singleton) usando Responses API
│   ├── openaiConfig.js           # Leitura de env vars, defaults, feature flags
│   ├── openaiSchemas.js          # Schemas JSON para respostas estruturadas
│   ├── openaiPayloads.js         # Builders de prompts
│   └── openaiDocumentExtract.js  # Extração de itens de PDF/DOCX/TXT via IA
│
├── rfq/                          # Núcleo de negócio (tudo que não é IA direta)
│   ├── parser.js                 # Entrada: parseRfqFromUrl, parseExcelToCanonical
│   ├── pipeline.js               # parseWithPipeline — orquestra o parse
│   ├── columnMapping.js          # Mapeamento fuzzy de colunas
│   ├── extractQuotationId.js     # Deduz ID da cotação
│   │
│   ├── io/
│   │   └── readWorkbook.js       # Leitura robusta de planilhas
│   ├── ingest/
│   │   ├── detectDocumentType.js # PDF/DOCX/TXT/XLSX
│   │   ├── extractPlainText.js   # Extrai texto de PDF/DOCX
│   │   └── legacyFromAiDocument.js
│   ├── normalize/
│   │   ├── money.js              # R$ 1.234,56 → 1234.56
│   │   ├── quantities.js         # "10 un", "2x50" → número + unidade
│   │   ├── boolean.js            # Sim/Não, S/N, etc.
│   │   └── canonical.js          # Estrutura canônica final
│   ├── templates/
│   │   ├── router.js             # Roteia para o parser correto
│   │   └── groupedBlocksParser.js# Parser específico para planilhas com vários fornecedores em blocos
│   ├── compare/
│   │   └── rank.js               # Ranking determinístico de fornecedores
│   ├── validate/
│   │   └── rules.js              # Regras de validação
│   ├── config/
│   │   └── aliases.js            # Dicionário de aliases de colunas
│   │
│   └── batch/                    # TUDO relacionado a compare-batch
│       ├── compareBatch.js       # Orquestrador principal do lote
│       ├── extractSupplierQuotes.js
│       ├── consolidateQuotes.js  # Agrupa itens por item_key entre fornecedores
│       ├── itemKey.js            # Chave canônica de item
│       ├── uncategorizedMerge.js # Mescla itens sem categoria
│       ├── semanticItemMatch.js  # [IA OPCIONAL] Equivalência semântica de itens
│       ├── semanticTelemetry.js  # Métricas do match semântico
│       ├── aiComparisonFeedback.js # Mensagem ao usuário quando IA foi pedida mas falhou
│       ├── reviewSummary.js      # Monta resumo de revisão manual
│       ├── batchDecision.js      # Applied/rejected decisions
│       ├── batchArtifactService.js # Regenera XLSX após decisão
│       ├── validateBatch.js      # Limites, MIME, tamanho
│       ├── batchResponse.js      # shapeCompareBatchResponse (slim vs debug)
│       ├── batchTypes.js         # BATCH_ERROR_CODES, enums
│       ├── batchMetrics.js       # Contadores/timers
│       ├── batchReadiness.js     # /ready checks
│       ├── batchStructuredLog.js # Logger estruturado JSON
│       ├── batchHistoryStore.js  # Persistência de histórico
│       ├── batchSnapshotStore.js # Persistência de snapshots
│       ├── batchDownloadStore.js # Tokens de download
│       ├── batchExportCleanup.js # Limpeza periódica dos XLSX
│       ├── batchRetentionCleanup.js # TTL de histórico/snapshots
│       ├── xlsxSchema.js         # Schema das abas exportadas
│       ├── workbookExport.js     # Gera o XLSX final
│       ├── pieChart3dPng.js      # Gráfico de pizza 3D embutido no XLSX
```

```
│       ├── supplierWinColors.js  # Esquema de cores
│       ├── jsonFileUtils.js      # Helpers de I/O
│       │
│       ├── config/
│       │   └── batchInfraConfig.js # URLs públicas, paths
│       ├── middleware/
│       │   ├── apiKeyAuth.js     # Valida X-API-Key
│       │   ├── requestCorrelation.js # X-Request-ID
│       │   ├── rateLimit.js      # Rate limiter (factory)
│       │   ├── rateLimitStoreFactory.js
│       │   ├── memoryRateLimitStore.js
│       │   └── redisRateLimitStore.js # Preparado p/ Redis quando disponível
│       └── storage/
│           ├── batchStorageContracts.js
│           ├── batchStorageFactory.js  # Escolhe local ou object storage
│           ├── localBatchExportAdapter.js
│           ├── localBatchHistoryAdapter.js
│           ├── localBatchSnapshotAdapter.js
│           └── objectBatchExportAdapter.js # S3/R2 compatível
│
├── test/                         # Testes (node:test). Cobertura: parser, batch, IA mockada, export
│
├── docs/                         # Documentação de contrato e batch
│   ├── README.md
│   ├── integration-schema.md
│   ├── batch-observability.md
│   ├── batch-xlsx-export.md
│   └── document-ingest.md
│
├── data/
│   └── batch-history/.gitkeep   # Fallback de histórico quando rfq/data/ não existe
│
└── rfq/data/                    # (RUNTIME, ignorado pelo git)
    ├── batch-history/           # JSON por batchId
    ├── batch-snapshots/         # Snapshots periódicos
    └── downloads/               # XLSX gerados com TTL
```

---

4. Fluxos de dados da aplicação

4.1. Parse de um arquivo (POST /api/parse)

```
Cliente                  server.js                   rfq/                    ai/
  |                          |                         |                       |
  |-- multipart file -------->|                         |                       |
  |                          |-- parseWithPipeline --->|                       |
  |                          |                         |-- detectDocumentType  |
  |                          |                         |-- readWorkbook        |
  |                          |                         |-- templates/router    |
  |                          |                         |-- normalize/*         |
  |                          |                         |-- validate/rules      |
  |                          |                         |                       |
  |                          |                         |-- [se flag] ---------> analyticSummary
  |                          |                         |<--------------------| JSON {summary}
  |                          |<------------------------|                       |
  |<-- JSON canonical_quot ---|                         |                       |
```

4.2. Comparação em lote (POST /api/compare-batch)

```
Cliente                    server.js                     rfq/batch/                  ai/
  |                            |                             |                          |
  |- multipart N arquivos ----->|                             |                          |
  |                            |-- rateLimitCompare          |                          |
  |                            |-- validateBatch             |                          |
  |                            |-- runCompareBatch ---------->|                          |
  |                            |                             |-- parse cada arquivo     |
  |                            |                             |-- extractSupplierQuotes  |
  |                            |                             |-- consolidateQuotes      |
  |                            |                             |                          |
  |                            |                             |-- [flag] --------------->|semanticItemMatch
  |                            |                             |<-------------------------|
  |                            |                             |-- rank determinístico    |
  |                            |                             |-- buildReviewSummary     |
  |                            |                             |                          |
  |                            |                             |-- [flag] --------------->|analyticSummary
  |                            |                             |<-------------------------|
  |                            |                             |-- exportBatchWorkbook    |
  |                            |                             |-- persist history        |
  |                            |<----------------------------|                          |
  |<-- JSON + download link ----|                             |                          |
  |                            |                             |                          |
  |-- GET /downloads/xxx.xlsx ->|                             |                          |
  |                            |-- validateDownloadToken     |                          |
  |<-- arquivo --- stream ------|                             |                          |
```

4.3. Decisão manual sobre um lote

```
POST /api/compare-batch/:id/decision  { status: "approved" | "rejected", note }
   -> rateLimitMutate
   -> requireBatchApiKey (se BATCH_API_KEY configurada)
   -> batchDecision.applyDecision
   -> batchArtifactService.applyDecisionToExport  (regenera XLSX final)
   -> persiste em histórico
```

---

5. Integração com OpenAI (determinístico vs IA)

Princípio do projeto: **a IA nunca substitui o motor determinístico**. Ela complementa.

Caso de uso	Onde é disparado	Flag que habilita	Fallback se falhar
Resolução de ambiguidade de layout	`rfq/pipeline.js`	`OPENAI_ENABLE_AMBIGUITY_RESOLUTION`	Usa heurística default, retorna warning
Resumo analítico sobre o resultado	`rfq/batch/compareBatch.js` após ranking	`OPENAI_ENABLE_ANALYTIC_SUMMARY`	Resultado numérico vai sem resumo + `ai_comparison_feedback`
Equivalência semântica de itens	`rfq/batch/semanticItemMatch.js`	`OPENAI_ENABLE_SEMANTIC_ITEM_MATCH` + `OPENAI_SEMANTIC_ITEM_MATCH_THRESHOLD`	Usa apenas `item_key` determinístico
Extração de itens de PDF/DOCX/TXT	`ai/openaiDocumentExtract.js`	Ativa por padrão quando há chave; `OPENAI_ENABLE_DOCUMENT_INGEST=0` desliga	415 Unsupported Media Type

Modelo usado: `gpt-4o-mini` (barato, bom para extração estruturada). Configurável via `OPENAI_MODEL`.

Formato de resposta: sempre JSON estruturado (Responses API com schema). Evita parse de texto livre.

Resiliência: `OPENAI_TIMEOUT_MS=30000`, `OPENAI_MAX_RETRIES=1`. Se estourar, o fluxo determinístico segue e o usuário recebe `ai_comparison_feedback` explicando.

6. Infraestrutura de produção

```
    Desenvolvedor (Windows PC)
           |
           |  git push origin main
           v
+-----------------------------+
|  GitHub                     |
|  TecnologiaDreamFactory/    |
|  suprimentos-dream (público)  |
+----------------+-------------+
                 |
                 |  webhook push
                 v
+-----------------------------+
|  Railway                    |
|  Projeto "suprimentos-dream"  |
|                             |
|  Build: Nixpacks detecta    |
|    Node → npm ci → npm start  |
|                             |
|  Runtime:                   |
|    - container Linux        |
|    - $PORT injetada         |
|    - 14 env vars OPENAI_*   |
|      + NODE_ENV=production  |
|    - filesystem efêmero     |
+----------------+-------------+
                 |
                 v
+-----------------------------+
|  Domínio público            |
|  *.up.railway.app  (TLS auto) |
+-----------------------------+
```

Componente	Detalhe
Build	Nixpacks (detecção automática de Node.js)
Comando de install	`npm ci` (se houver `package-lock.json`) ou `npm install`
Comando de start	`npm start` → `node server.js` (vindo do `package.json`)
Porta	`$PORT` injetada pelo Railway; `server.js` lê `process.env.PORT`
TLS	Certificado automático do Railway
Escala	1 réplica por padrão (ajustável no plano Pro)
Redeploy	Automático a cada push em `main`

7. Pipeline CI/CD (como uma alteração chega em produção)

```
1.  Dev edita arquivo em branch local
        v
2.  npm test   (opcional mas recomendado)
        v
3.  git add -A
    git commit -m "feat/fix/chore: descrição"
        v
4.  git push origin <branch>
        v
5.  Abre Pull Request no GitHub → main
        v
6.  Code review + merge (squash)
        v
7.  GitHub envia webhook → Railway
        v
8.  Railway inicia novo build
        - Nixpacks resolve dependências
        - npm ci
        - Empacota container
        v
9.  Railway deploy
        - Sobe novo container
        - Health-check na PORT
        - Swap atômico com o antigo
        v
10. Novo commit está em produção
    URL pública continua a mesma
```

Tempo total típico: 2 a 4 minutos da pressão de "merge" até estar no ar.

Rollback: aba *Deployments* no Railway → selecionar um deploy antigo → *Redeploy*. Em 30 segundos o estado anterior volta.

---

8. Variáveis de ambiente

8.1. Obrigatórias

Nome	Usado em	Observação
`OPENAI_API_KEY`	Toda a camada `ai/`	Sem ela, app roda 100% determinístico (funciona, mas sem resumos/semântica)

8.2. Recomendadas em produção

Nome	Valor típico	Efeito
`NODE_ENV`	`production`	Otimizações do Express (cache de views, mensagens de erro enxutas)
`PORT`	**não definir manualmente**	Railway injeta

8.3. Controle de IA

Nome	Padrão	Para que serve
`OPENAI_MODEL`	`gpt-4o-mini`	Modelo usado em todas as chamadas
`OPENAI_TIMEOUT_MS`	`30000`	Timeout de cada chamada
`OPENAI_MAX_RETRIES`	`1`	Retries em caso de falha transitória
`OPENAI_TEMPERATURE`	`0.2`	Baixa para favorecer consistência
`OPENAI_ENABLE_AMBIGUITY_RESOLUTION`	`true`	Resolve ambiguidade de layout na pipeline
`OPENAI_ENABLE_ANALYTIC_SUMMARY`	`true`	Gera resumo textual do resultado de lote
`OPENAI_PARSING_CONFIDENCE_THRESHOLD`	`0.75`	Só aceita sugestão da IA acima desse valor

8.4. Equivalência semântica de itens

Nome	Padrão	Para que serve
`OPENAI_ENABLE_SEMANTIC_ITEM_MATCH`	`true`	Liga/desliga match semântico
`OPENAI_SEMANTIC_ITEM_MATCH_THRESHOLD`	`0.80`	Acima disso: aceita como match
`OPENAI_SEMANTIC_ITEM_MATCH_LOW_CONFIDENCE`	`0.60`	Abaixo disso: descarta
`OPENAI_SEMANTIC_ITEM_MATCH_MAX_CANDIDATES`	`5`	Máx. de candidatos comparados
`OPENAI_SEMANTIC_ITEM_MATCH_MAX_CALLS`	`10`	Máx. de chamadas à OpenAI por lote

8.5. Downloads e batch (opcionais)

Nome	Efeito
`BATCH_API_KEY`	Se definida, rotas `/api/compare-batch/*` exigem `X-API-Key`
`BATCH_DOWNLOAD_REQUIRE_TOKEN`	Exige `?token=` no link de download
`BATCH_DOWNLOAD_TOKEN_TTL_MS`	TTL do token (padrão 15 min)
`BATCH_DOWNLOAD_CONSUME_TOKEN_ON_SUCCESS`	Link de uso único
`BATCH_RATE_LIMIT_COMPARE_MAX`	Rate limit do `/api/compare-batch` (padrão 30/min)
`BATCH_RATE_LIMIT_MUTATE_MAX`	Rate limit de decisões/mutations (padrão 20/min)
`REDIS_URL`	Se definida, rate-limit passa a usar Redis em vez de memória

8.6. Ingestão de documentos

Nome	Efeito
`OPENAI_ENABLE_DOCUMENT_INGEST=0`	Desliga extração via IA de PDF/DOCX/TXT
`OPENAI_DOCUMENT_INGEST_MAX_CHARS`	Limite de texto enviado à IA (padrão 120000)
`OPENAI_DOCUMENT_INGEST_TIMEOUT_MS`	Timeout específico (padrão 60000)

---

9. Rotas HTTP expostas

Método	Rota	Autenticação	Descrição
GET	`/`	—	Serve `public/index.html`
GET	`/ping`	—	Liveness trivial
GET	`/health`	—	Status simples
GET	`/ready`	—	Readiness: verifica acesso a diretórios, OpenAI opcional
POST	`/api/parse`	—	Upload multipart de 1 arquivo → parse
POST	`/rfq/parse`	—	Parse por URL externa (JSON: `{ file_url, rfq_id, ... }`)
POST	`/api/compare`	—	Comparação legada
POST	`/api/compare-batch`	rate-limit + API key (se configurada)	Upload de N arquivos → lote completo
GET	`/api/compare-batch/:batchId`	API key	Histórico do lote
POST	`/api/compare-batch/:batchId/decision`	rate-limit + API key	Aprovar/rejeitar e regerar XLSX
GET	`/downloads/:fileName`	token opcional + API key	Baixa XLSX gerado

Request ID: todo request recebe `X-Request-ID` (middleware `requestCorrelation`), útil para rastrear nos logs.

10. Segurança (camadas aplicadas)

Camada	Implementação	Onde
TLS	Termina no edge do Railway	infra
CORS	`cors()` aberto (projeto serve seu próprio frontend)	`server.js`
Tamanho de body	`multer memoryStorage` (configurar limite em produção)	`server.js`
Rate limiting	Por IP/namespace, janelas configuráveis	`rfq/batch/middleware/rateLimit.js`
API key de batch	Header `X-API-Key` validado	`middleware/apiKeyAuth.js`
Download tokens	UUIDs com TTL e one-shot opcional	`batchDownloadStore.js`
Path traversal	Validação do nome no endpoint `/downloads/:fileName`	`server.js` linhas 135–166
Injection de IDs	Regex `^B-\d+-[a-f0-9]+$` para `batchId`	`server.js`
Segredos	Só em env vars; `.gitignore` protege `.env` , `*.pem` , `credentials.json`	`.gitignore`
Dependências	`package-lock.json` fixado; `npm ci` em produção	Railway build

Ação pendente recomendada para o time: ativar regra de proteção do branch `main` no GitHub exigindo PR + review.

11. Persistência e armazenamento

11.1. Situação atual no Railway

O filesystem do container **é efêmero**: a cada redeploy/reinício, diretórios como `rfq/data/batch-history/`, `rfq/data/batch-snapshots/` e `rfq/data/downloads/` são apagados.

11.2. Opções para persistir

Opção	Passos	Custo adicional
Volume do Railway	*Settings* → *Volumes* → + *New* apontando para `/app/rfq/data`	incluso no plano Hobby
Object Storage (S3/R2)	Ativar `objectBatchExportAdapter.js` via variáveis (ver `batchStorageFactory.js`)	centavos/mês
Redis (rate-limit distribuído)	Adicionar plugin Redis no Railway e setar `REDIS_URL`	custo do plugin

11.3. Por que ainda não é crítico

- Histórico é apenas "nice to have" — nenhuma operação depende de lote anterior.
- XLSX gerados têm TTL curto (minutos) e são recriados sob demanda.
- Para **MVP**, o volume nativo do Railway resolve. Para escala, migrar exports para S3 usando o adapter já pronto.

---

12. Observabilidade (logs, métricas, readiness)

12.1. Logs estruturados

`rfq/batch/batchStructuredLog.js` emite JSON com:

- `event`, `route`, `request_id`, `batch_id`
- `status`, `http_status`
- `execution_ms`

Visualização: aba *Deployments* → *View Logs* no Railway, ou via `railway logs` no CLI.

12.2. Métricas operacionais

`rfq/batch/batchMetrics.js` mantém contadores em memória (por processo):

- Total de lotes, sucessos, falhas
- Tempo médio de execução
- Chamadas à OpenAI (sucesso/erro/fallback)

Exportação a Prometheus/Datadog ainda não implementada (item de evolução).

12.3. Readiness

`GET /ready` devolve:

- `disk_ok`: diretórios acessíveis
- `openai_configured`: booleano
- `dependencies_loaded`: checagem dos módulos críticos

Pode ser usado pelo Railway (ou healthcheck externo) para load balancer.

13. Testes automatizados

Arquivo	Cobre
`test/pipeline.test.js`	Pipeline de parse completo
`test/columnMapping.test.js`	Fuzzy matching de colunas
`test/groupedBlocksParser.test.js`	Layout "grouped"
`test/extractQuotationId.test.js`	Regex de ID
`test/batch.test.js`	Fluxo batch completo
`test/batch-hardening.test.js`	Edge-cases de batch
`test/batch-workflow-hardening.test.js`	Decisões, aprovação
`test/batch-production-middleware.test.js`	Middlewares (auth, rate-limit)
`test/batch-storage-adapters.test.js`	Adapters local/object
`test/batchObservability.test.js`	Logger estruturado, métricas
`test/semanticItemMatch.test.js`	Match semântico (IA mockada)
`test/semanticTelemetry.test.js`	Contadores de match
`test/documentIngest.test.js`	PDF/DOCX/TXT
`test/reviewSummary.test.js`	Montagem do resumo
`test/uncategorizedMerge.test.js`	Mescla de itens sem categoria
`test/openai.test.js`	Cliente OpenAI (mockado)
`test/aiComparisonFeedback.test.js`	Mensagens ao usuário

Executar:

```
npm test
```

Runner nativo (`node --test`). Sem Jest, sem dependências extra.

14. Fluxo de desenvolvimento recomendado para o time

```
main (produção)  <-- protegido, só recebe PR
    ^
    |  squash & merge
    |
feature/<descricao>
bugfix/<ticket>
chore/<descricao>
```

Passo a passo de uma entrega

```
1. git checkout main
2. git pull
3. git checkout -b feature/minha-mudanca
4. (codar)
5. npm test                       # local
6. git add -A
7. git commit -m "feat: resumo curto"
8. git push -u origin feature/minha-mudanca
9. gh pr create --fill            # ou abrir pela web
10. Revisão de outro dev
11. Merge (squash) em main
12. Railway redeploy automático (2–4 min)
13. Validar no domínio público
```

Padrão de commit

Seguir uma variante de Conventional Commits:

- `feat:` nova funcionalidade
- `fix:` correção de bug
- `chore:` manutenção (deps, configs)
- `docs:` só documentação
- `refactor:` sem mudar comportamento
- `test:` só testes
- `perf:` otimização

Revisão de código

- **Nunca** mergear sem rodar `npm test`.
- Conferir se novas env vars foram documentadas em `.env.example` e no `GUIA_DEPLOY.md`.
- Conferir se alterações em `rfq/batch/` preservam contrato (campos de `shapeCompareBatchResponse`).

---

15. Riscos conhecidos e evoluções sugeridas

Risco / lacuna	Impacto	Evolução sugerida
Filesystem efêmero no Railway	Histórico some em redeploys	Volume do Railway (30 min de esforço)
CORS totalmente aberto	Qualquer domínio pode chamar a API	Restringir a uma allowlist quando houver frontends separados
Rate limit em memória	Não compartilha entre réplicas	Plugar Redis (`redisRateLimitStore.js` já pronto)
Sem métricas externalizadas	Dashboard depende de logs	Exportar para Prometheus ou Datadog
Sem alerta de erro	Falha em produção só é vista se alguém abrir os logs	Integrar Sentry ou Rollbar
`BATCH_API_KEY` opcional	Endpoints públicos em produção	Tornar obrigatória quando o sistema for aberto a usuários
Chave OpenAI única	Um vazamento = comprometimento total	Rotacionar periodicamente; usar chaves distintas por ambiente
Sem GitHub Actions	Testes rodam só localmente antes de push	Adicionar workflow `on: pull_request` rodando `npm test`
Branch `main` sem proteção	Push direto pode derrubar produção	Habilitar *Require pull request* em *Settings → Branches*
Dependências não monitoradas	CVEs não detectadas automaticamente	Ativar Dependabot no GitHub (gratuito)

Apêndice A — Resumo para reunião executiva

- **O que é:** sistema que lê planilhas de cotação de fornecedores, compara e ranqueia, exportando XLSX consolidado com ajuda opcional de IA para interpretar itens ambíguos.
- **Arquitetura:** monolito Node.js + Express, sem banco de dados, frontend estático.
- **Hospedagem:** Railway (Nixpacks) com deploy automático a cada push em `main` no GitHub.
- **Tempo de publicar uma mudança:** ~3 minutos do merge até produção.
- **Custo operacional:** US$ 5 a 10 por mês (Railway Hobby + uso da OpenAI).
- **Capacidade atual:** 1 réplica, rate-limit de 30 lotes/min por IP. Suficiente para equipe de compras.
- **Para escalar:** adicionar volume (persistência), Redis (rate-limit distribuído), múltiplas réplicas (plano Pro do Railway).
- **Dependência externa crítica:** OpenAI API. Queda dela degrada funcionalidades, mas não derruba a aplicação.

Última atualização deste documento: 21/04/2026 **Responsável técnico:** Almir Almeida (`TecnologiaDreamFactory` no GitHub)